

# Redes de Computadores: Internet

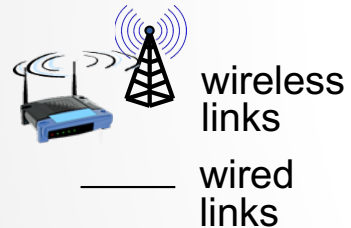
Revisão dos conceitos

# What's the Internet: “nuts and bolts” view

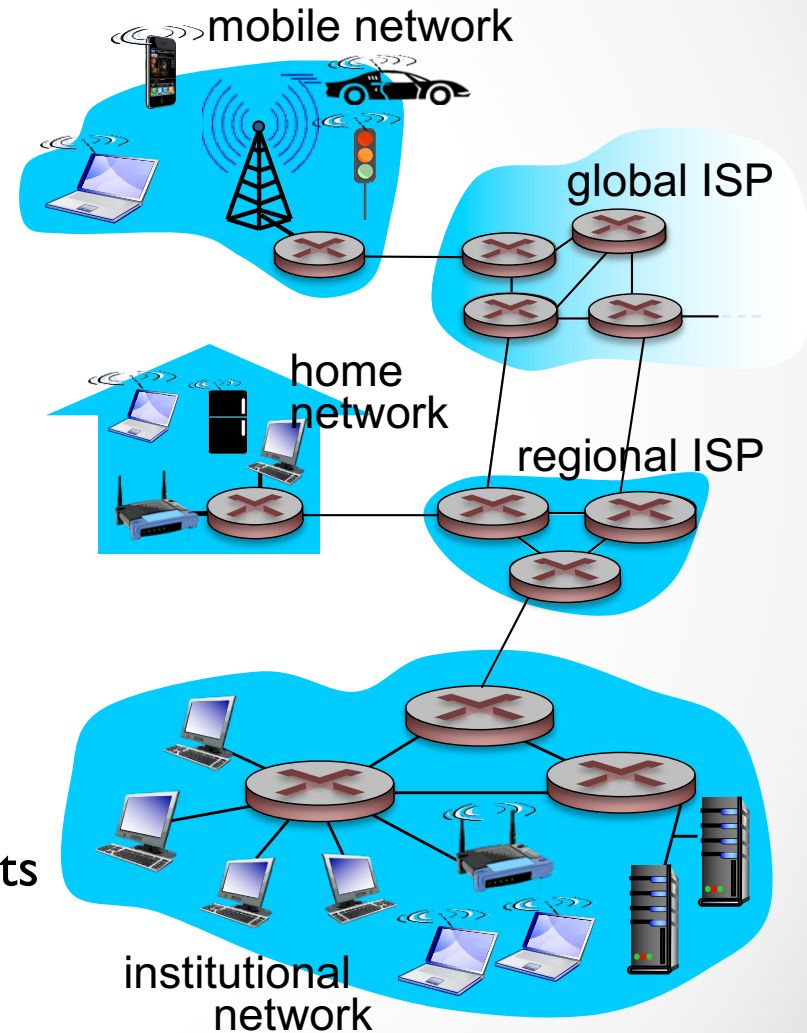


- billions of connected computing devices:

- *hosts* = *end systems*
- running *network apps*

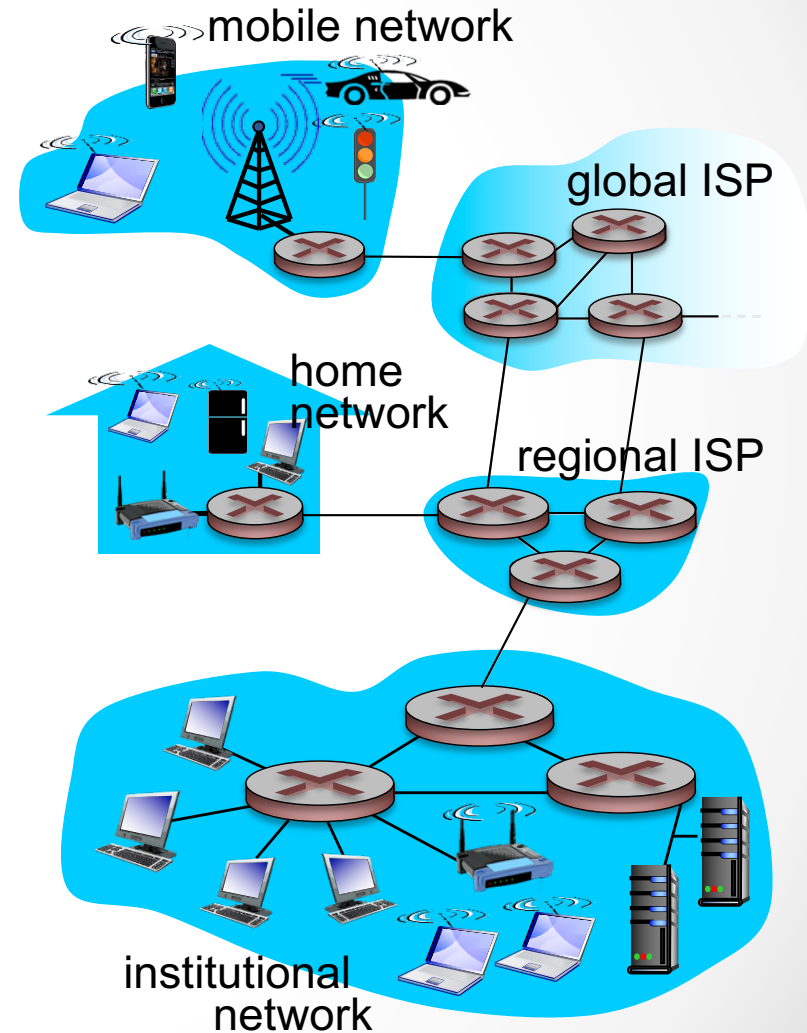


- *communication links*
  - fiber, copper, radio, satellite
  - transmission rate: *bandwidth*
- *packet switches*: forward packets (chunks of data)
  - *routers* and *switches*



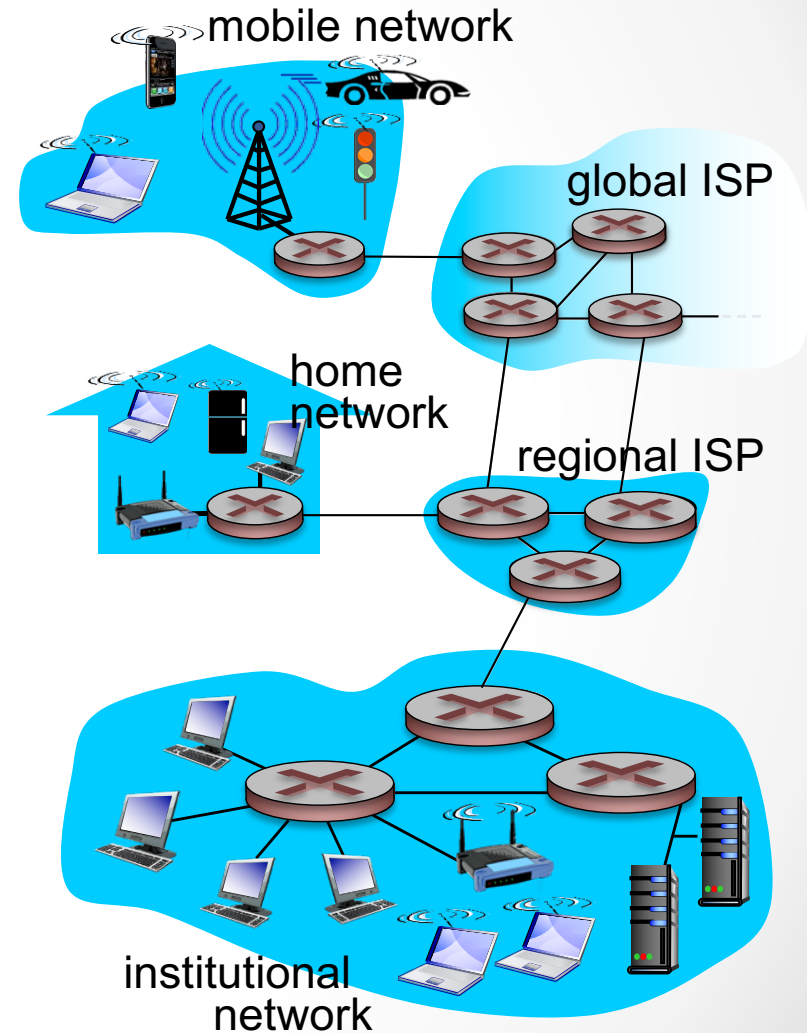
# What's the Internet: “nuts and bolts” view

- *Internet*: “network of networks”
  - Interconnected ISPs
- *protocols* control sending, receiving of messages
  - e.g., TCP, IP, HTTP, Skype, 802.11
- *Internet standards*
  - RFC: Request for comments
  - IETF: Internet Engineering Task Force



# What's the Internet: a service view

- *infrastructure that provides services to applications:*
  - Web, VoIP, email, games, e-commerce, social nets, ...
- *provides programming interface to apps*
  - hooks that allow sending and receiving app programs to “connect” to Internet
  - provides service options, analogous to postal service



# Protocol “layers”

*Networks are complex,  
with many “pieces” :*

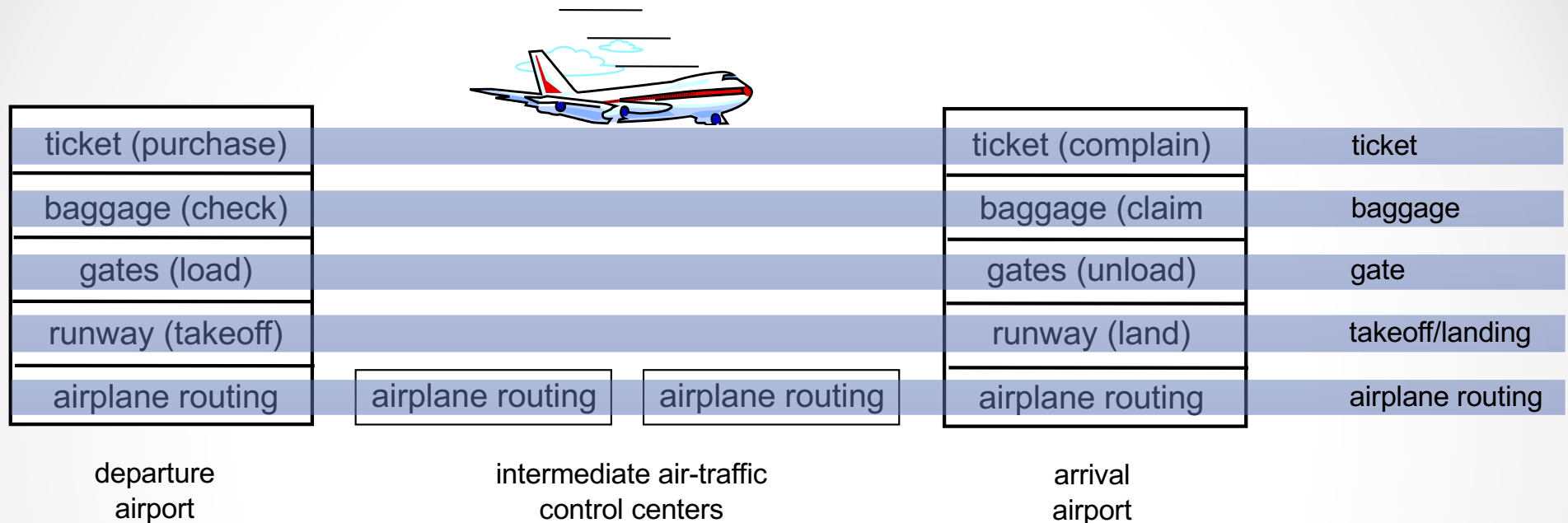
- hosts
- routers
- links of various media
- applications
- protocols
- hardware, software

## *Question:*

is there any hope of *organizing*  
structure of network?

.... or at least our discussion of  
networks?

# Layering of airline functionality



**layers:** each layer implements a service

- via its own internal-layer actions
- relying on services provided by layer below

# Why layering?

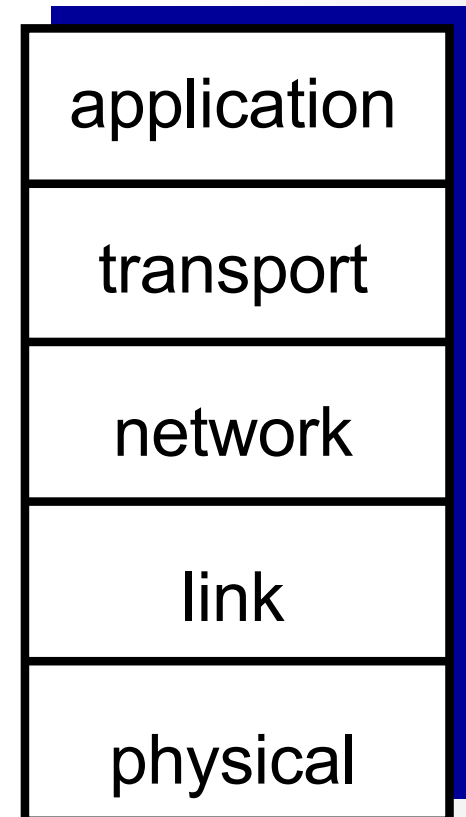
dealing with complex systems:

- explicit structure allows identification, relationship of complex system's pieces
  - layered **reference model** for discussion
- modularization eases maintenance, updating of system
  - change of implementation of layer's service transparent to rest of system
  - e.g., change in gate procedure doesn't affect rest of system
- layering considered harmful?

# Internet protocol stack

---

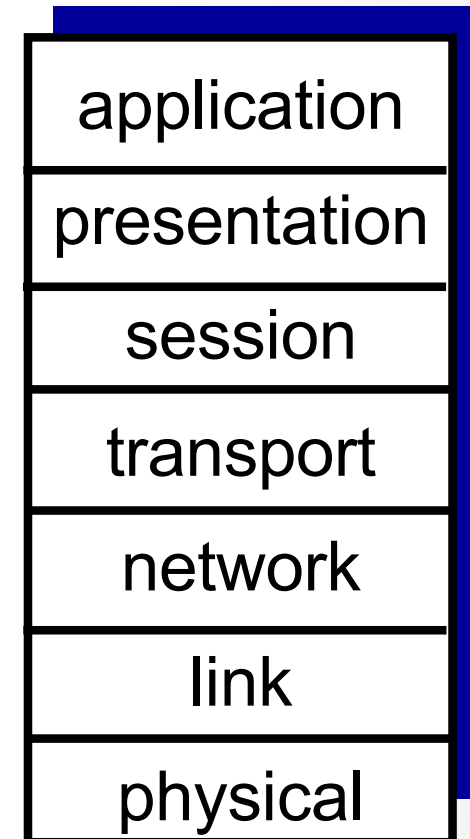
- *application*: supporting network applications
  - FTP, SMTP, HTTP
- *transport*: process-process data transfer
  - TCP, UDP
- *network*: routing of datagrams from source to destination
  - IP, routing protocols
- *link*: data transfer between neighboring network elements
  - Ethernet, 802.111 (WiFi), PPP
- *physical*: bits “on the wire”



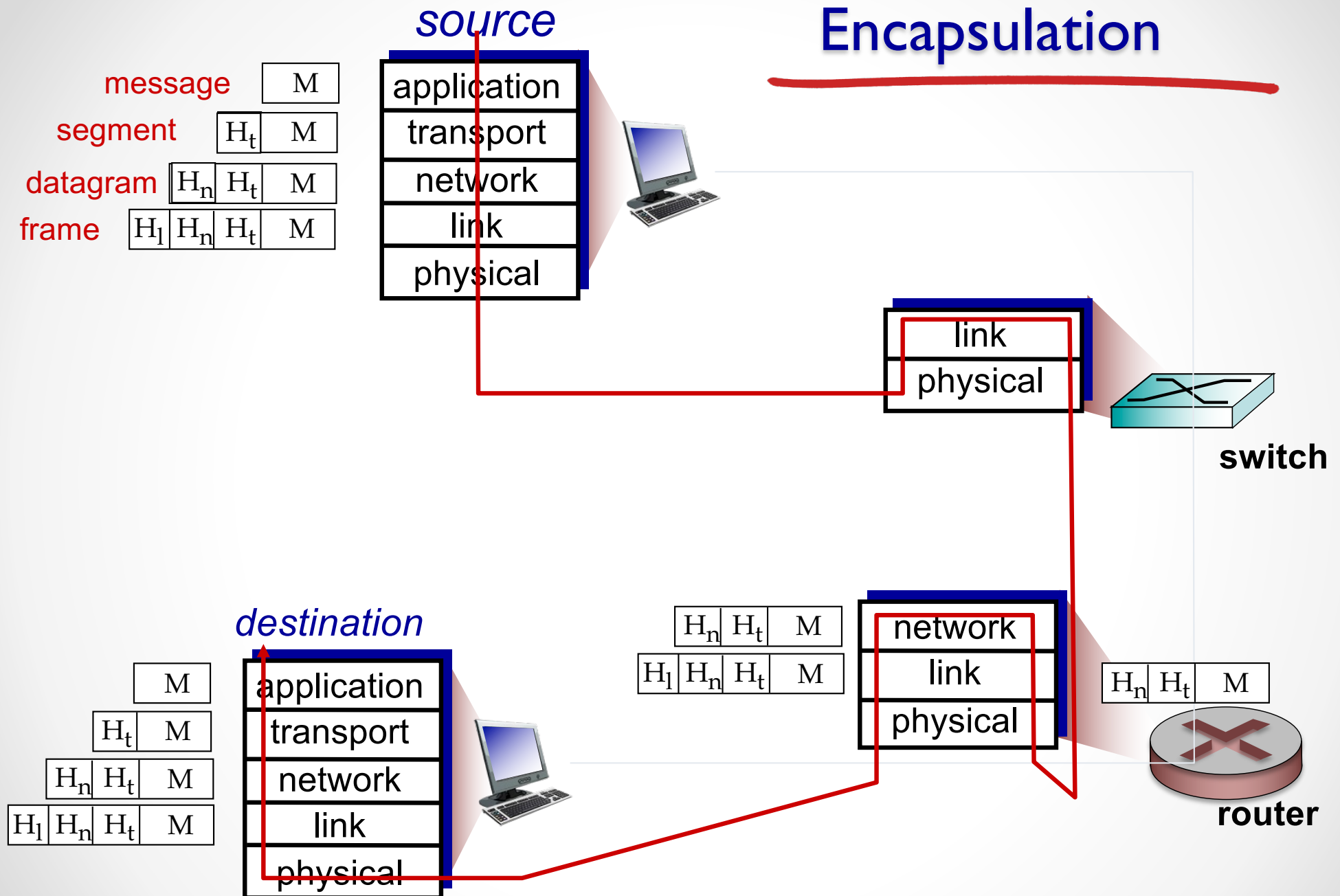
# ISO/OSI reference model

---

- **presentation**: allow applications to interpret meaning of data, e.g., encryption, compression, machine-specific conventions
- **session**: synchronization, checkpointing, recovery of data exchange
- Internet stack “missing” these layers!
  - these services, *if needed*, must be implemented in application
  - needed?

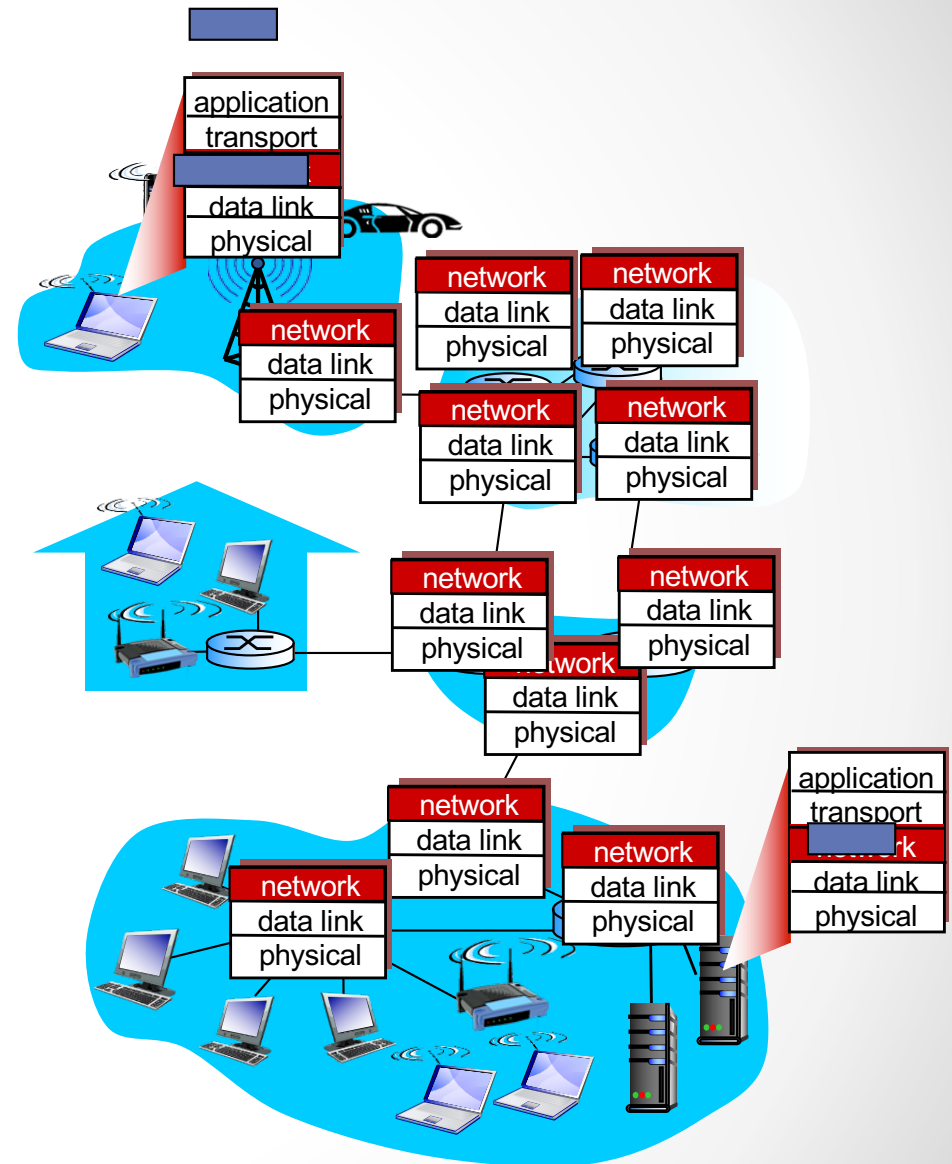


# Encapsulation



# Network layer

- transport segment from sending to receiving host
- on sending side encapsulates segments into datagrams
- on receiving side, delivers segments to transport layer
- network layer protocols in **every** host, router
- router examines header fields in all IP datagrams passing through it



# Network-layer functions

*Recall: two network-layer functions:*

- *forwarding*: move packets from router's input to appropriate router output *data plane*
- *routing*: determine route taken by packets from source to destination *control plane*

*Two approaches to structuring network control plane:*

- per-router control (traditional)
- logically centralized control (software defined networking)

# Two key network-layer functions

---

## *network-layer functions:*

- *forwarding*: move packets from router's input to appropriate router output
- *routing*: determine route taken by packets from source to destination
  - *routing algorithms*

## *analogy: taking a trip*

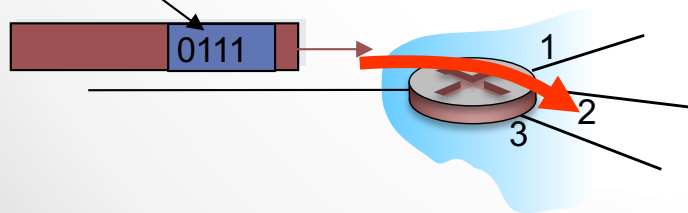
- *forwarding*: process of getting through single interchange
- *routing*: process of planning trip from source to destination

# Network layer: data plane, control plane

## Data plane

- local, per-router function
- determines how datagram arriving on router input port is forwarded to router output port
- forwarding function

values in arriving packet header

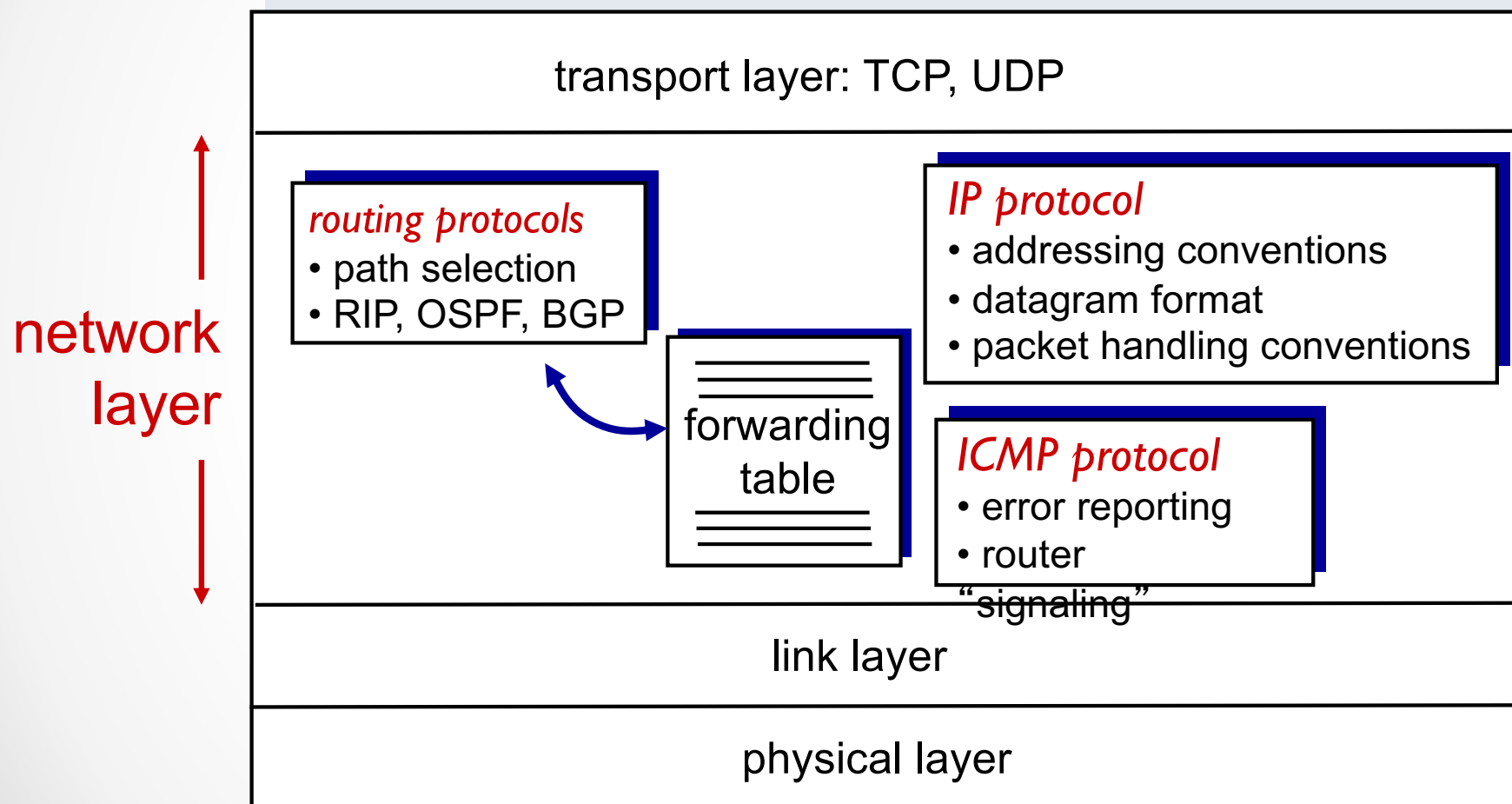


## Control plane

- network-wide logic
- determines how datagram is routed among routers along end-end path from source host to destination host
- two control-plane approaches:
  - *traditional routing algorithms*: implemented in routers
  - *software-defined networking (SDN)*: implemented in (remote) servers

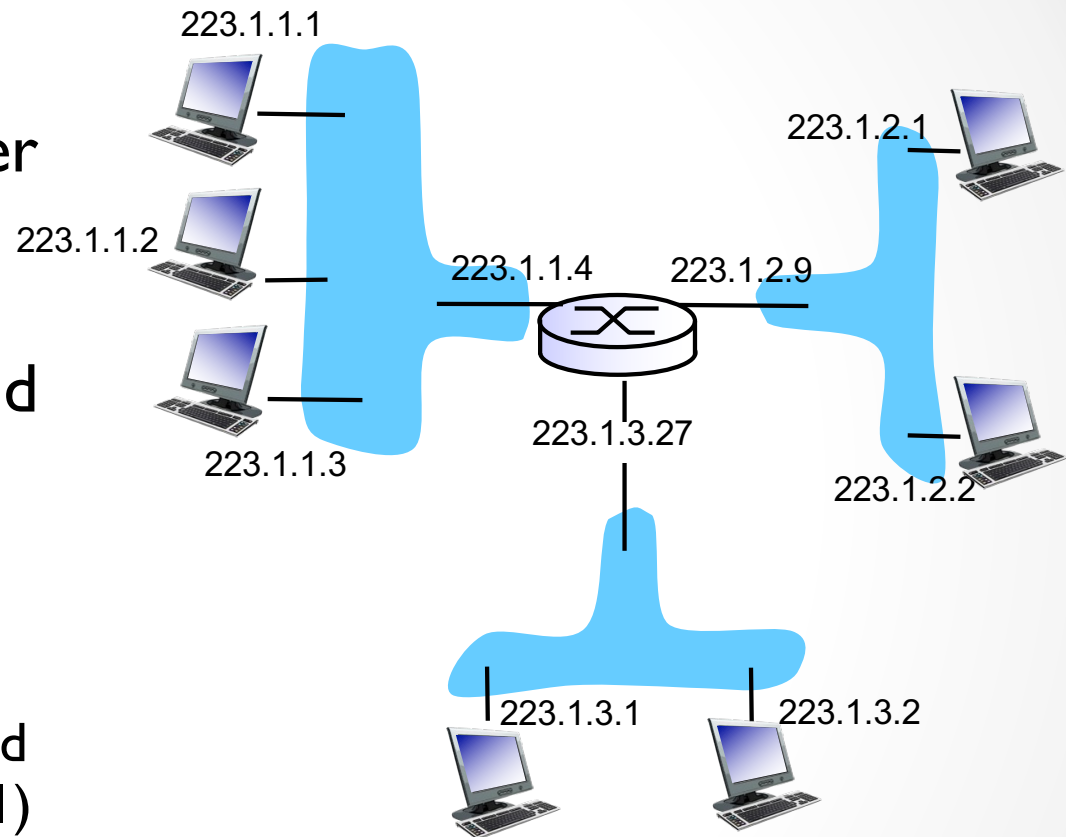
# The Internet network layer

host, router network layer functions:



# IP addressing: introduction

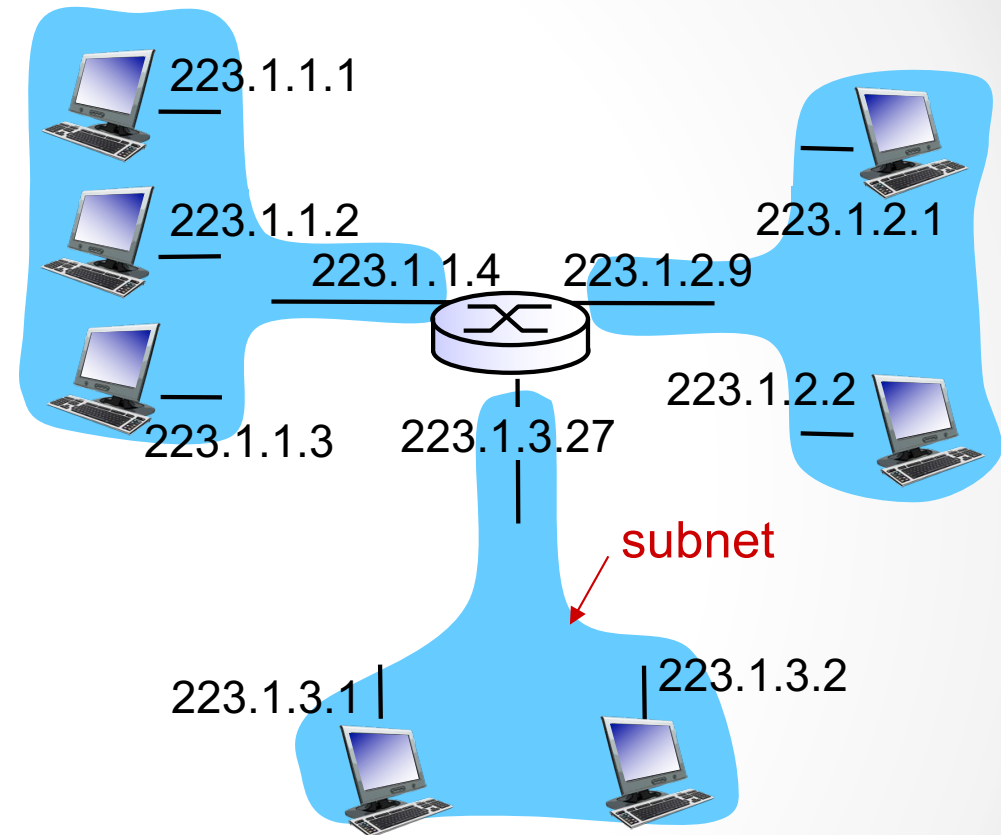
- **IP address:** 32-bit identifier for host, router interface
- **interface:** connection between host/router and physical link
  - router's typically have multiple interfaces
  - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)
- **IP addresses associated with each interface**



$$223.1.1.1 = \underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$$

# Subnets

- IP address:
  - subnet part - high order bits
  - host part - low order bits
- *What's a subnet ?*
  - device interfaces with same subnet part of IP address
  - can physically reach each other *without intervening router*



network consisting of 3 subnets

# Routing protocols

**Routing protocol goal:** determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- path: sequence of routers packets will traverse in going from given initial source host to given final destination host
- “good”: least “cost”, “fastest”, “least congested”
- routing: a “top-10” networking challenge!

# Making routing scalable

our routing study thus far - idealized

- all routers identical
- network “flat”

... *not* true in practice

*scale*: with billions of destinations:

- can't store all destinations in routing tables!
- routing table exchange would swamp links!

*administrative autonomy*

- internet = network of networks
- each network admin may want to control routing in its own network

# Internet approach to scalable routing

aggregate routers into regions known as “autonomous systems” (AS) (a.k.a. “domains”)

## intra-AS routing

- routing among hosts, routers in same AS (“network”)
- all routers in AS must run *same* intra-domain protocol
- routers in *different* AS can run *different* intra-domain routing protocol
- gateway router: at “edge” of its own AS, has link(s) to router(s) in other AS'es

## inter-AS routing

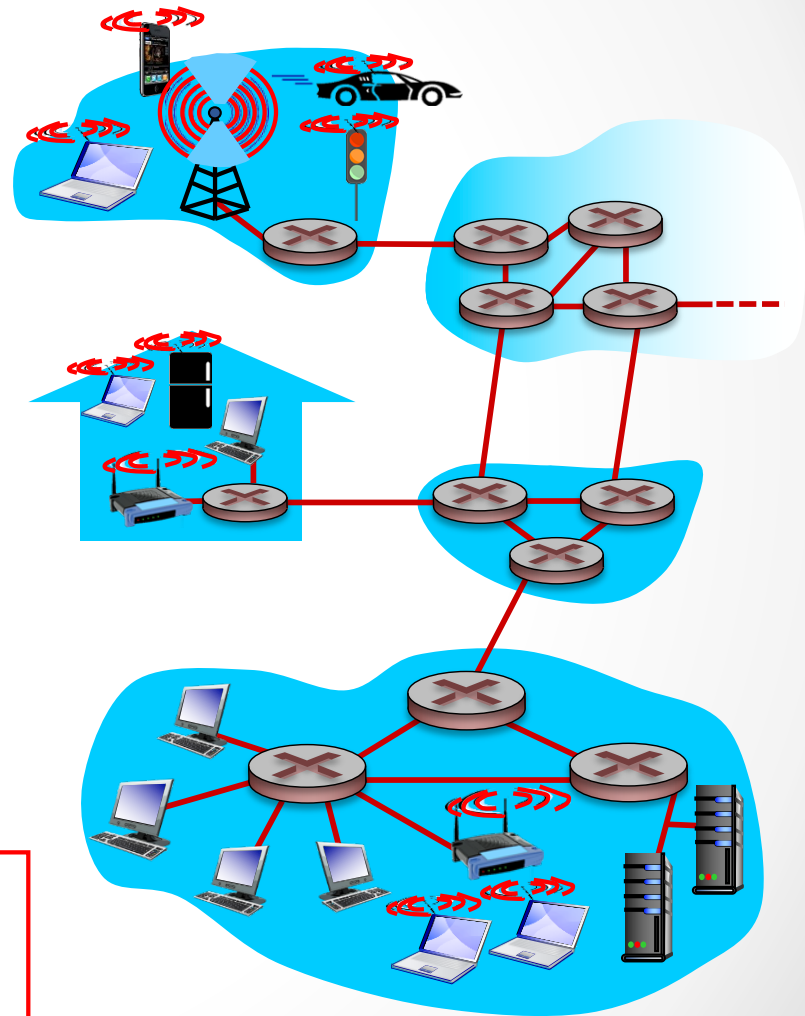
- routing among AS'es
- gateways perform inter-domain routing (as well as intra-domain routing)

# Link layer: introduction

## *terminology:*

- hosts and routers: **nodes**
- communication channels that connect adjacent nodes along communication path: **links**
  - wired links
  - wireless links
  - LANs
- layer-2 packet: **frame**, encapsulates datagram

*data-link layer* has responsibility of transferring datagram from one node to *physically adjacent* node over a link



# Link layer: context

- datagram transferred by different link protocols over different links:
  - e.g., Ethernet on first link, frame relay on intermediate links, 802.11 on last link
- each link protocol provides different services
  - e.g., may or may not provide rdt over link

## *transportation analogy:*

- trip from Princeton to Lausanne
  - limo: Princeton to JFK
  - plane: JFK to Geneva
  - train: Geneva to Lausanne
- tourist = **datagram**
- transport segment = **communication link**
- transportation mode = **link layer protocol**
- travel agent = **routing algorithm**

# Link layer services

---

- *framing, link access:*
  - encapsulate datagram into frame, adding header, trailer
  - channel access if shared medium
  - “MAC” addresses used in frame headers to identify source, destination
    - different from IP address!
- *reliable delivery between adjacent nodes*
  - seldom used on low bit-error link (fiber, some twisted pair)
  - wireless links: high error rates
    - Q: why both link-level and end-end reliability?

# Link layer services

---

- *flow control:*
  - pacing between adjacent sending and receiving nodes
- *error detection:*
  - errors caused by signal attenuation, noise.
  - receiver detects presence of errors:
    - signals sender for retransmission or drops frame
- *error correction:*
  - receiver identifies *and corrects* bit error(s) without resorting to retransmission
- *half-duplex and full-duplex*
  - with half duplex, nodes at both ends of link can transmit, but not at same time

# MAC protocols: taxonomy

---

three broad classes:

- *channel partitioning*
  - divide channel into smaller “pieces” (time slots, frequency, code)
  - allocate piece to node for exclusive use
- *random access*
  - channel not divided, allow collisions
  - “recover” from collisions
- *“taking turns”*
  - nodes take turns, but nodes with more to send can take longer turns

# MAC addresses and ARP

- 32-bit IP address:
  - *network-layer* address for interface
  - used for layer 3 (network layer) forwarding
- MAC (or LAN or physical or Ethernet) address:
  - function: *used ‘locally’ to get frame from one interface to another physically-connected interface (same network, in IP-addressing sense)*
  - 48 bit MAC address (for most LANs) burned in NIC ROM, also sometimes software settable
  - e.g.: 1A-2F-BB-76-09-AD

/

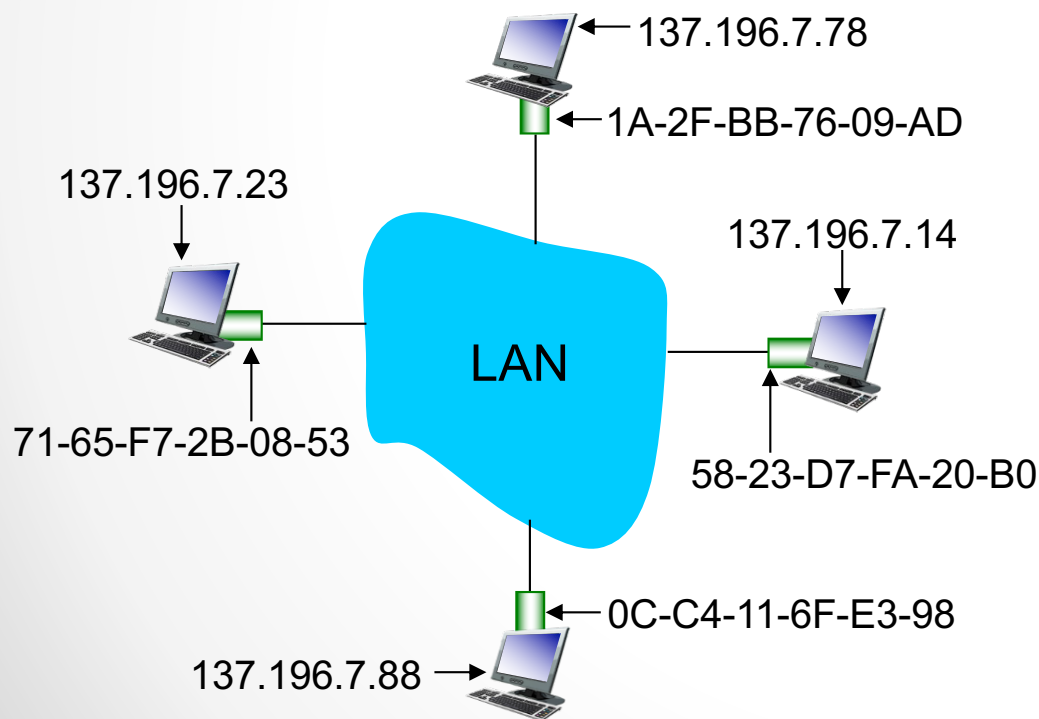
hexadecimal (base 16) notation  
(each “numeral” represents 4 bits)

# ARP: address resolution protocol

**Question:** how to determine interface's MAC address, knowing its IP address?

**ARP table:** each IP node (host, router) on LAN has table

- IP/MAC address mappings for some LAN nodes:  
**< IP address; MAC address; TTL >**
- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)



# Ethernet CSMA/CD algorithm

---

1. NIC receives datagram from network layer, creates frame
2. If NIC senses channel idle, starts frame transmission. If NIC senses channel busy, waits until channel idle, then transmits.
3. If NIC transmits entire frame without detecting another transmission, NIC is done with frame !
4. If NIC detects another transmission while transmitting, aborts and sends jam signal
5. After aborting, NIC enters *binary (exponential) backoff*:
  - after  $m$ th collision, NIC chooses  $K$  at random from  $\{0, 1, 2, \dots, 2^m - 1\}$ . NIC waits  $K \cdot 512$  bit times, returns to Step 2
  - longer backoff interval with more collisions

# Referências

- Livro do Kurose, capítulos 1, 2, 3, 4, 5 e 6.
- Consulta de artigos:
  - <http://citeseer.ist.psu.edu>
  - <http://ieeexplore.ieee.org>
- Bibliografia adicional: Ler artigo do Jacobson - **Congestion avoidance and control**

# Exercício

- Em uma janela dos ou shell, execute o comando:
  - `tracert/traceroute www.mit.edu`
- Analise a resposta
- Agora instale o analisador de protocolo: Wireshark ([www.wireshark.org](http://www.wireshark.org))
- Agora analise novamente a resposta ao comando `traceroute` via analisador de protocolo.
- Veja como é implementado o comando `traceroute` (através de mensagens ICMP) em todas as camadas.

# ICMP: internet control message protocol

- used by hosts & routers to communicate network-level information

- error reporting: unreachable host, network, port, protocol
- echo request/reply (used by ping)

- network-layer “above” IP:

- ICMP msgs carried in IP datagrams

- **ICMP message:** type, code plus first 8 bytes of IP datagram causing error

| <u>Type</u> | <u>Code</u> | <u>description</u>                            |
|-------------|-------------|-----------------------------------------------|
| 0           | 0           | echo reply (ping)                             |
| 3           | 0           | dest. network unreachable                     |
| 3           | 1           | dest host unreachable                         |
| 3           | 2           | dest protocol unreachable                     |
| 3           | 3           | dest port unreachable                         |
| 3           | 6           | dest network unknown                          |
| 3           | 7           | dest host unknown                             |
| 4           | 0           | source quench (congestion control - not used) |
| 8           | 0           | echo request (ping)                           |
| 9           | 0           | route advertisement                           |
| 10          | 0           | router discovery                              |
| 11          | 0           | TTL expired                                   |
| 12          | 0           | bad IP header                                 |

# Traceroute and ICMP

- source sends series of UDP segments to destination
  - first set has TTL = 1
  - second set has TTL=2, etc.
  - unlikely port number
- when datagram in  $n$ th set arrives to  $n$ th router:
  - router discards datagram and sends source ICMP message (type 11, code 0)
  - ICMP message include name of router & IP address

- when ICMP message arrives, source records RTTs

## *stopping criteria:*

- UDP segment eventually arrives at destination host
- destination returns ICMP “port unreachable” message (type 3, code 3)
- source stops

